

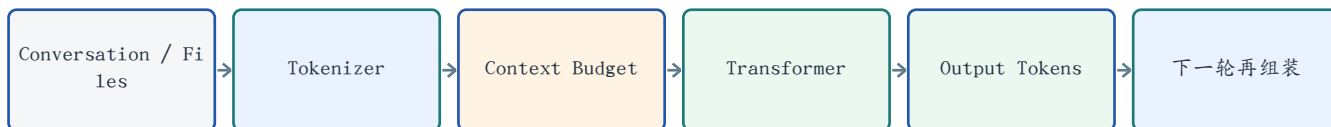
第21讲

Context Window：上下文窗口到底是什么？

为什么 32K、128K、1M 不等于模型拥有长期记忆？

本讲主线

Context Window 是一次推理时模型当前可见的 token 工作区；它决定“这次能同时看到多少内容”，但不等于跨会话持久记忆。



应用层常常需要在每次调用前重新组装上下文

学习定位：把“长上下文”“记忆”“RAG”“历史消息”“KV Cache”这些容易混在一起的概念彻底分层。

1. 先回答最核心的问题：Context Window 是什么？

Context Window（上下文窗口）可以先理解成：模型在一次推理过程中允许同时放进工作区的 token 范围。

如果一个模型的上下文窗口是 N tokens，那么系统需要在这个总预算内安排系统提示、用户消息、历史对话、工具定义、检索资料、工具结果，以及通常还要给即将生成的输出留出空间。

一个非常实用的近似式：

$$\text{context budget} \approx \text{input tokens} + \text{output tokens}$$

注意

具体 API 对“最大上下文”“最大输入”“最大输出”的限制方式可能不同，所以工程上必须看具体模型文档；但概念上都在做同一件事：有限 token 工作区的预算分配。

2. 32K、128K、1M 到底表示什么？

这里的 K、M 是 token 数量级，不是汉字数，也不是单词数。

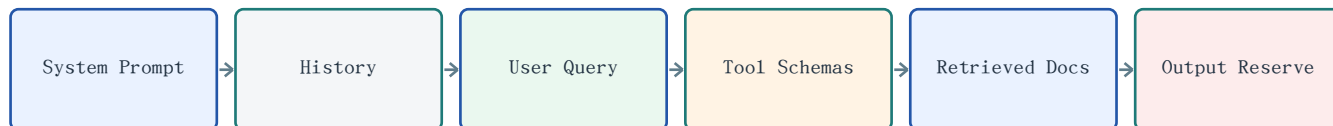
标记	大致含义	不能直接等同于
32K	约 3.2 万 token 的上下文容量	3.2 万汉字 / 3.2 万单词
128K	约 12.8 万 token	12.8 万段文本
1M	约 100 万 token	模型永久记住 100 万 token

同一篇中文、英文、代码文档被 tokenizer 切分后的 token 数可能差异很大，所以“能放多少页 PDF” 只能做粗略估计。

与第2讲连接	Context Window 的单位之所以是 token，是因为 Transformer 接收的本来就不是“页”和“字”，而是 tokenizer 生成的 token 序列。
--------	---

3. 一次真实请求里，谁会占用 Context?

很多初学者只看到自己最后输入的一句话，但真正送入模型的 prompt 往往远比这长。



System Prompt: 角色、规则、格式要求。

Conversation History: 为了让模型“记得前文”，应用通常会把旧消息重新放进去。

Tool Schemas: 函数名、参数说明也会变成 token。

RAG / Files: 检索出的文档片段会进入当前上下文。

Tool Results: 数据库、网页、代码执行结果也可能很长。

Output Reserve: 还需要给本轮生成留下空间。

4. Context ≠ Memory：最重要的区分

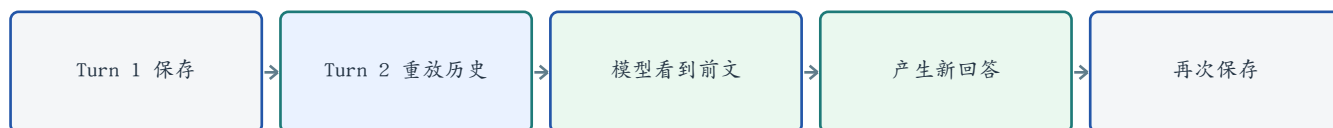
概念	本质	是否跨调用天然存在
Context	本次推理当前输入的 token 工作区	否；下一次通常要重新构造
Model Parameters	训练得到的权重	存在，但不是本次对话日志
KV Cache	本次生成过程中缓存历史 K/V	通常只服务当前推理会话/缓存策略
External Memory	数据库、向量库、文件、事件日志	可以长期保存，但需再次检索进入 Context

所以：

Context Window = 工作台大小；Long-term Memory = 仓库；Retrieval = 从仓库把需要的东西搬回工作台。

5. 为什么聊天看起来像“模型记住了我”？

最常见的机制不是模型权重在每轮对话后都被重新训练，而是应用层保存历史，然后在下一次请求时重新组装。



因此，很多“短期记忆”的本质是：

runtime state + context reconstruction

关键

如果旧消息没有被重新放进上下文，也没有通过外部记忆检索回来，模型通常不会凭空知道那段旧消息。

6. Context Window 满了以后会发生什么？

当系统要发送的 token 超过允许窗口时，应用必须采取某种策略。常见做法包括：

- 拒绝请求或报 context length exceeded;

- 截断最旧消息 (sliding window) ;

- 先把旧历史摘要成更短文本;

- 只保留最近若干轮;

- 从长期记忆中按需检索相关片段，而不是全部塞进去;

- 对工具输出进行压缩、抽取关键字段。

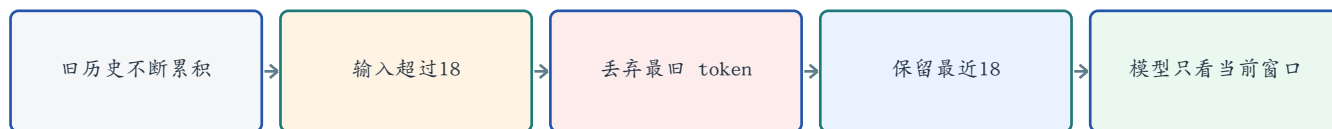
这意味着：

历史“存在于系统里” != 当前模型“现在看得到”。

7. 一个极小例子：固定 18 token 输入预算

假设总窗口 24 个 toy tokens，并预留 6 个给输出，那么当前输入只能使用 18 个。

对话继续增长时，如果采用最简单的 sliding window:



这不是“模型突然失忆”，而是那段信息已经不在这一次 forward 的输入张量里。

本讲实验

材料包里的 Python 脚本会把每一轮 total history、kept context、dropped tokens 全部打印出来。

8. 长 Context 为什么不是免费的？

上下文越长，通常会同时增加计算、显存和延迟压力。

资源	为什么变大	直觉
Prefill Compute	需要处理更多输入 token	先读完更长的材料
Attention Work	标准全注意力会产生大量 token-pair 交互	每个位置要和更多位置建立关系
KV Cache	每层要保存更多历史 K/V	桌面越长，便签越多
Latency	更长输入常提高首 token 等待	先把更厚的材料读进去
Cost	很多 API 按输入/输出 token 计费	上下文变长直接增加账单

9. 从 Attention 角度看：为什么标准长上下文很贵？

在最朴素的 full self-attention 中，长度为 n 的序列需要形成一个 $n \times n$ 的 attention score 矩阵。

score matrix shape = $[n, n]$

因此 token 数从 n 翻倍到 $2n$, pairwise score 数量会从 n 增长到 $4n$ 。

但要谨慎

现代模型可能使用 FlashAttention、GQA、滑动窗口、稀疏注意力、分块策略等降低实际常数或改变部分复杂度；所以“所有长上下文都严格按同一种 n 成本增长”不能机械套用。

10. KV Cache 为什么也会随着上下文变长?

第19讲学过: Decode 时为了不重复计算过去 token 的 K/V, 系统会缓存它们。

一个简化的直觉式:

$$\text{KV cache size} \propto \text{layers} \times \text{sequence length} \times \text{KV heads} \times \text{head dimension}$$

因此上下文越长, 缓存的历史 K/V 通常越多。GQA / MQA 的一个重要推理优势, 就是减少需要保存的 K/V head 数。

不要混淆

KV Cache

是推理加速缓存, 不是用户长期记忆数据库。关闭会话、切换服务或缓存被清理后, 它不能替代 Memory。

11. “窗口能放下” ≠ “模型能同样好地使用所有位置”

Context capacity 只回答：技术上允许多少 token 进入一次请求。它不自动保证：模型对窗口里每一个位置都拥有同样稳定的检索和推理能力。

问题	含义
Capacity	能不能把这么多 token 放进去
Retrieval	能不能从很长文本中找到关键片段
Reasoning	找到以后能不能正确组合和推导
Robustness	关键片段换位置、加干扰以后是否仍稳定
Efficiency	为了这一能力要付出多少延迟/显存/成本

研究长上下文时，这五件事最好分开测。

12. Lost in the Middle：为什么中间的信息可能更难用？

长上下文研究中一个经典现象是：当关键证据被放在很长输入的不同位置时，模型表现可能随位置变化；一些模型对开头或结尾的信息利用更稳定，而中间位置可能更弱。

这类现象常被概括为 Lost in the Middle。

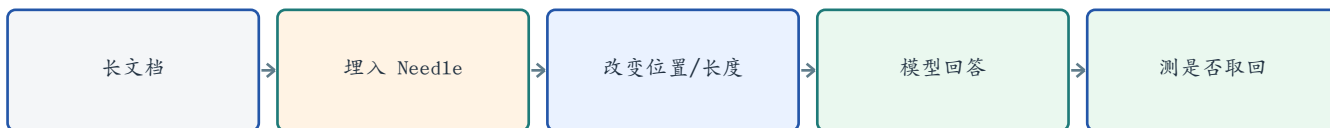
正确理解

它不是“所有模型、所有任务都必然在正中间失败”的物理定律，而是一类需要通过位置控制实验验证的长上下文可靠性问题。

所以一个严谨实验可以把同一证据分别放在：开头 / 1/4 / 中间 / 3/4 / 结尾，再测准确率。

13. Needle-in-a-Haystack 测试能说明什么？

一个常见长上下文测试方式是把一条“针”（关键事实）埋进大量无关文本“干草堆”里，再让模型找回来。



它适合测试 retrieval，但不能单独证明复杂长上下文 reasoning 已经解决。

例子

模型能从 500K token 中找到“密码是 8472”，不代表它一定能跨 500K token 组合 20 个分散事实完成多步推理。

14. Long Context ≠ RAG：两个不同思路

方案	核心做法	优势	风险
Long Context	把很多资料直接放进 prompt	简单、端到端可见	成本高、干扰多、利用率不一定稳定
RAG	先检索少量相关片段再放入 prompt	节省上下文、聚焦相关证据	检索阶段可能漏掉关键资料

实际系统常常混合使用：长窗口作为容量上限，RAG 负责在进入窗口前筛选信息。

大仓库 + 大桌面，并不等于把整个仓库每天全倒在桌面上。

15. Context Engineering：比“把更多东西塞进去”更重要

当模型本身已经足够强时，系统性能往往取决于“当前上下文里到底放什么、按什么结构放、什么时候删”。

Selection: 只选择与当前任务有关的信息。

Ordering: 重要规则、证据、当前任务放在合适位置。

Compression: 长工具输出变成结构化摘要。

Retrieval: 需要时再从长期记忆取回。

State: 把稳定状态单独维护，不靠整段聊天历史反复传递。

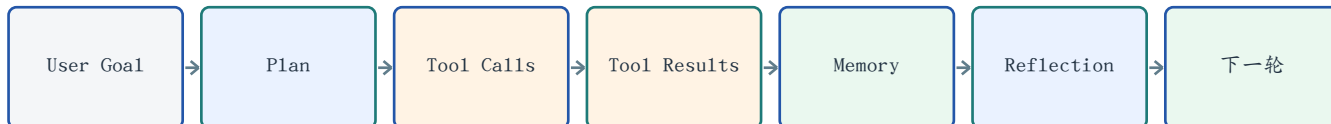
Verification: 检查摘要或检索是否丢掉关键约束。

科研视角

Context Engineering 本质上是在一个有限信息带宽系统里做信息选择、压缩、调度和恢复。

16. Agent 为什么特别容易 Context Bloat?

Agent Runtime 会不断产生新信息，导致上下文膨胀。



典型来源包括：

- 很长的 system prompt 和 tool descriptions;

- 每一步 action / observation 全部保留;

- 网页、终端日志、代码、JSON 结果直接原样回填;

- Reflection / Critique 每轮继续累积;

- 多个 Agent 把彼此的完整历史互相广播。

最终可能出现：token 成本上升、延迟上升、注意力被无关信息稀释、关键约束被淹没。

17. Context Bloat 的一个系统学视角

把 Agent 的上下文看成一个容量有限的通信信道：

$$\text{useful information density} = \text{useful task-relevant tokens} / \text{total context tokens}$$

如果总 token 一直增长，但有用信息比例下降，那么“上下文更长”反而可能让系统效率更差。

可以研究的指标

Context utilization、有效信息密度、冗余率、检索命中率、token cost、latency、critical fact retention，都可以成为 Agent Systems 的实验变量。

18. 多 Agent 系统中的 Context 更复杂

多 Agent 不只是“每个 Agent 有一个窗口”。通信本身也会产生额外 token。

通信策略	Context 代价	信息风险
Full Broadcast	所有 Agent 看所有消息	token 爆炸、重复严重
Summary Broadcast	广播压缩摘要	摘要可能丢信息
Selective Routing	只把相关消息发给相关 Agent	需要路由机制
Shared Memory	把状态写到公共存储，按需读取	需要解决一致性和检索问题

因此多 Agent 的规模扩大，不仅是 model call scaling，也可能是 context / communication scaling。

19. 一个研究设计：测 Context 长度，不要只测“最大能塞多少”

如果以后研究 Agent 长上下文可靠性，可以设计一个非常标准的实验矩阵：

自变量	水平示例	要观察什么
Context Length	8K / 32K / 128K	性能、成本、TTFT
Evidence Position	开头 / 中间 / 结尾	位置敏感性
Distractor Ratio	低 / 中 / 高	抗干扰能力
Compression	无 / 摘要 / 结构化	信息损失与成本
Retrieval	无 / Top-k / 分层检索	召回与最终任务表现
关键	不要把“支持 128K”直接当成“128K 场景可靠”。真正的科研问题是：在什么条件下可靠、哪里开始退化、为什么退化。	

20. Context 与 Long-term Memory 应怎样协作？



一个成熟 Agent 系统通常需要区分：

Working Context: 眼下任务真正需要的内容。

Episodic Memory: 过去发生过什么。

Semantic Memory: 抽取出的稳定事实或知识。

State Store: 结构化任务状态、变量、进度。

Retrieval Policy: 什么时候、取哪一条、取多少。

所以“给模型一个超长窗口”不能替代完整的 Memory Architecture。

21. Summarization Memory：压缩有代价

一种常见做法是把旧历史摘要成更短文本，再把摘要保留在上下文里。

优点	代价
显著节省 token	摘要本身可能丢掉边界条件
保留高层任务脉络	错误摘要会持续污染后续轮次
减少重复工具日志	难以恢复被丢弃的细节
更适合长期对话	需要决定何时重新摘要
科研问题	摘要不是纯压缩算法，它会改变后续 Agent 可观察到的状态。因此可以研究 compression error 如何传播成 planning / tool-use error。

22. 常见误区

误区	更准确的理解
1M Context = 永久记忆1M token	只表示单次/当前推理可用窗口数量级，不等于跨会话持久记忆
窗口越大越好	更大提供容量，但也带来成本、干扰、检索与鲁棒性问题
模型能放下就一定能用好	容量与有效利用是两个不同指标
KV Cache 就是 Memory	它是本次推理的 K/V 计算缓存
RAG 被超长 Context 淘汰	RAG 仍能做选择、聚焦和成本控制
多 Agent 共享全部历史最安全	可能造成 context bloat、重复与错误扩散

23. 配套实验：Context Budget 到底怎样“挤满”？

材料包中的 04_context_window_budget_demo.py 使用一个很小的教学窗口：

window = 24 toy tokens, output reserve = 6, input budget = 18

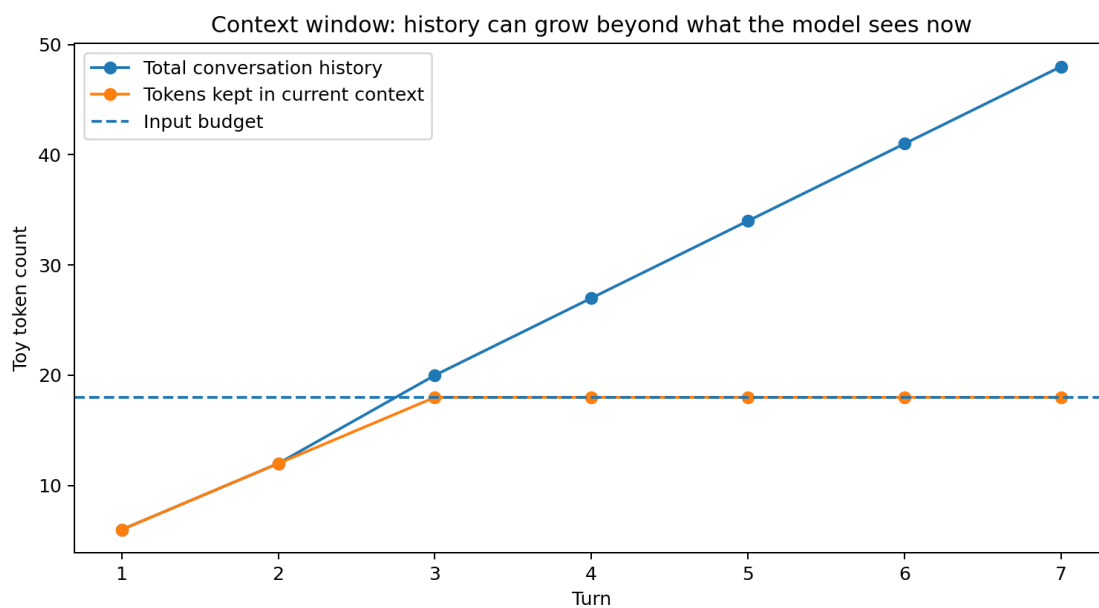
脚本会不断追加 system / user / assistant / tool 消息，然后打印：

总历史 token 数；

当前实际保留进 prompt 的 token 数；

因为窗口限制被丢弃的最旧 token 数；

当前模型真正“看得到”的可见上下文。



24. 实验第二部分：为什么 Retrieval 比“全塞进去”更像长期记忆

脚本还放了 5 条外部 memory，其中只有部分与“重要实验规则”有关。一个极简 overlap scorer 会先给 memory 排序，再把最相关的一条取回来。

这虽然不是实际 embedding retrieval，但它体现了正确的系统结构：



真正的长期记忆系统不是让模型永远背着全部历史，而是：

store more, retrieve less, use only what matters now

25. 你可以亲手改哪些参数？

把 WINDOW 从 24 改成 16，看旧消息更早被裁掉；

把 OUTPUT_RESERVE 从 6 改成 10，观察可用输入预算进一步缩小；

把 tool result 改成长日志，观察它怎样挤占用户历史；

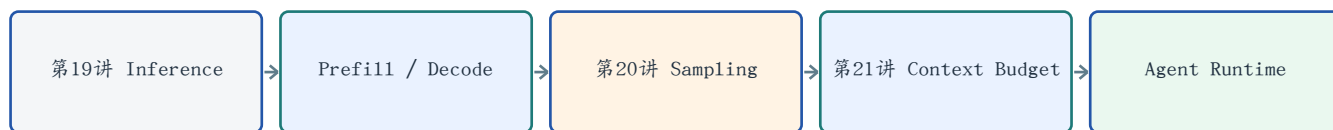
把“只保留最近 token”改成“永远保留 system + 最近消息”，比较两种策略；

给 memory 增加错误但关键词很多的条目，观察简单检索器为什么可能取错。

目的

不要把这个脚本当成真实 LLM tokenizer，而是用它建立“上下文是有限资源，需要调度”的系统直觉。

26. 把第19-21讲串起来



现在你已经能把一次 Agent LLM 调用看成一个资源受限的系统过程：先组装 Context，再 Prefill，再逐 token Decode，每一步还要做 sampling；如果 Agent 循环继续，新的 observation 又会进入下一轮 Context。

这为后面理解 RAG、Memory、Tool Use 和 Agent Runtime 打下了直接基础。

27. 本讲小结

Context Window 是一次推理当前可见的 token 工作区，不等于跨会话长期记忆。

系统提示、历史对话、工具说明、检索文档、工具结果和输出空间都在争夺同一 token 预算。

历史存在于应用层，不代表这一轮模型一定看得到；超窗后需要截断、摘要、检索或压缩。

长 Context 会增加 Prefill、Attention、KV Cache、延迟和成本压力。

“能放下”与“能可靠利用”必须分开评估；位置、干扰、推理深度都可能影响结果。

Long Context 与 RAG 不是二选一，成熟系统往往同时使用。

Agent 特别容易出现 Context Bloat；多 Agent 还会叠加通信 token。

长期记忆更适合“外部保存 + 按需检索 + 当前上下文使用”，而不是把全部历史永远塞进 prompt。

核心句：

Context

是工作台，不是仓库；窗口变大只是桌子变大，不代表你已经拥有了好的记忆系统。

28. 练习题

- 1) 为什么 128K Context 不能说成“模型记住 128K token”？
- 2) 一次 API 请求里，除了用户问题，还有哪些内容可能占用上下文？
- 3) 为什么 Context Window 满了以后，不一定是“模型本身失忆”？
- 4) Context、KV Cache、External Memory 三者分别是什么？
- 5) 为什么长上下文会增加 TTFT 和显存压力？
- 6) “模型支持 1M token”与“模型能在 1M token 内稳定找出证据并推理”有什么区别？
- 7) RAG 与 Long Context 为什么可以同时存在？
- 8) Agent 为什么比普通单轮聊天更容易 Context Bloat？
- 9) 多 Agent Full Broadcast 会带来哪些 context / communication 问题？
- 10) 设计一个实验，验证关键证据放在开头、中间、结尾是否影响 Agent 成功率。

答案见独立《练习题与答案》PDF。